

SCRIPTS

NETMAJ/SAUVEGARDE

Objectifs : créer deux scripts, un qui réalise les MAJ et nettoyage et un qui réalise les sauvegarde, avec option bouton sur lanceur. Les scripts devront être déployables à partir d'une clé USB

Pré-requis : Clé USB, OS Linux ou MAC

NOTIONS

OPTIMISATION

IPV6

Désactiver Ipv6 : `sysctl -w net.ipv6.conf.all.disable_ipv6=1`
`sysctl -w net.ipv6.conf.default.disable_ipv6=1`
`sysctl -w net.ipv6.conf.tun0.disable_ipv6=1`

Pour le réactiver (au cas où) : `sysctl -w net.ipv6.conf.all.disable_ipv6=0`
`sysctl -w net.ipv6.conf.default.disable_ipv6=0`
`sysctl -w net.ipv6.conf.tun0.disable_ipv6=0`
`sysctl -p`

SI SERVEUR OU PAS SERVEUR

On peut tenter de détecter l'adresse IP qui exécute le script, si \$IP="adresse serveur" **X**

VIM

DH -F

1 script mise à jour + nettoyage

Script demande : Mise à jour de serveur ou de client ?

- sil est à l-heure ✓
- paquet ntp installé ✓
- copier fichier passwd + tous les fichiers de conf /adm/sav ou bkp en local passwd.date coller avec la date
- si serveur : sauvegarder fichier dhcp.conf
- nettoyage (les deux)

Script hebdomadaire de mise à jour, nettoyage réalisé précédemment :

```
#!/bin/bash
```

```
echo "Lancement du script 1 - MAJ + nettoyage"
echo "Voici la date actuelle:"
date
```

```
sudo aptitude install -y vim ntpdate ntp
```

```
echo "synchronisation de l'heure"
```

```
# Sauvegarde du fichier de configuration original
cp /etc/ntp.conf /etc/ntp.conf.bak
```

```
# Activation et démarrage du service NTP
systemctl enable ntp
systemctl restart ntp
```

```
# Synchronisation immédiate de l'heure
```

```
systemctl stop ntp
```

```
ntpdate 0.fr.pool.ntp.org
```

```
systemctl start ntp
```

```
echo "Souhaitez-vous effectuer une mise à jour client ou serveur ? (client/serveur)"
```

```
read -r choix
```

```
case "${choix,,}" in
```

```
    client)
```

```
        echo "Mise à jour du client en cours..."
```

```
        sudo apt-get update
```

```
        sudo apt-get install -y aptitude
```

```
        aptitude update
```

```
        aptitude safen-upgrade
```

```
# Nettoyer les paquets en état défectueux + supprimer les paquets en  
dépendances orphelines
```

```
sudo aptitude -y purge ~abreak
```

```
sudo aptitude -y purge ~c
```

```
# Nettoyer les fichiers de paquets téléchargés mais non installés & supprimer les  
anciennes vers. des paquets
```

```
sudo aptitude clean
```

```
sudo aptitude autoclean
```

```
serveur)
```

```
    echo "Mise à jour du serveur en cours..."
```

```
    sudo apt-get update
```

```
    sudo apt-get install -y aptitude
```

```
    aptitude update
```

```
    aptitude safen-upgrade
```

```
    echo "Copie des fichiers de configuration..."
```

```
        cp /etc/resolv.conf /adm/sav/resolv.conf.$date
```

```
        cp /etc/passwd /adm/sav/resolv.conf.$date
```

```
        cp /etc/shadow /adm/sav/shadow.conf.$date
```

```
# Désactivation IPv6
```

```
touch /etc/sysctl.d/10-disable-ipv6.conf
```

```

cat > /etc/sysctl.d/10-disable-ipv6.conf << EOF
'net.ipv6.conf.all.disable_ipv6=1"
    net.ipv6.conf.default.disable_ipv6=1
    net.ipv6.conf.lo.disable_ipv6=1' | sudo tee
/etc/sysctl.d/10-disable-ipv6.conf
EOF
sudo sysctl -p /etc/sysctl.d/10-disable-ipv6.conf

```

trouver le script qui purge tous les noyaux (récupère le nom de la dernière kernel et degage tous ceux qui ne sont pas celle là)

MACHINES QUI VONT CHERCHER LES ORDRES / POUR MONTER LA TAILLE DES SWAP/
EN FONCTION DU TYPE D'OS - LSB RELEASE

1 script backup

-repertoire de sauvegarde crée
-fichier de sauvegarde

```
#!/bin/bash
```

```
# --- Configuration ---
```

```

DEST_IP="172.16.19.26" # Adresse IP du serveur distant
DEST_USER=$(whoami)    # Utilisateur distant (même nom qu'en local)
DEST_BASE_DIR="/Leray6/usr/users/nosav/$DEST_USER/" # Chemin distant

```

```
# Fichiers/dossiers à sauvegarder
```

```

SOURCES=(
    "$HOME/.bashrc"
    "$HOME/.bash_profile"
    "$HOME/.config/"
    "$HOME/Documents/"
)

```

```
# --- Vérifications ---
```

```

echo "Vérification de la connexion au serveur distant..."
if ! ping -c 1 -W 1 "$DEST_IP" &> /dev/null; then
    echo "Erreur : Serveur $DEST_IP inaccessible !"
    exit 1
fi

```

```

# --- Copie vers le serveur distant ---
echo "Début de la sauvegarde vers $DEST_IP..."
for item in "${SOURCES[@]}"; do
    if [ -e "$item" ]; then
        echo " → Envoi de $item..."
        scp -r "$item" "$DEST_USER@$DEST_IP:$DEST_BASE_DIR"
        if [ $? -ne 0 ]; then
            echo " /\ Échec de l'envoi pour $item"
        fi
    else
        echo " /\ $item non trouvé, ignoré."
    fi
done

echo "Transfert terminé vers $DEST_IP:$DEST_BASE_DIR"

```

Serveur backup : 14

Il va falloir faire une modif de script pour que ça sauvegarde le dossier agent

Comment faire depuis l'interface de linux mint pour créer un lanceur pour cliquer sur un bouton dans l'interface pour lancer mon script

Ordis sous mac, Windows, linux, pouvoir le lancer sans ouvrir un terminal, quand on veut ---->
documenter la procédure

LMJL sur le bureau (lanceur) qui demande le mot de passe de l'université
puisque ça va lancer sur Leray6

Sinon tâches programmées sur Windows

1 lanceur pour les buttons

Installer python et Tkinter : apt install python3-tk

Installer un environnement graphique sur la VM :

sudo apt update

sudo apt install xfce4 xfce4-goodies -y

Problème actuel : l'interface graphique ne fonctionne plus au bout d'un moment

Script en python : (abandonné)

```
import tkinter as tk
import subprocess
import os

def lancer_script(script_path):
    if script_path.endswith(".py"):
        subprocess.Popen(["python", script_path])
    else:
        subprocess.Popen(script_path, shell=True)

fenetre = tk.Tk()
fenetre.title("Mon Lanceur de Scripts")
fenetre.geometry("300x200")

script1 = os.path.abspath("script1.py")
script2 = os.path.abspath("script2.py")
script3 = os.path.abspath("script3.py")

btn1 = tk.Button(fenetre, text="Lancer Script 1", command=lambda: lancer_script(script1),
height=2, width=20)
btn2 = tk.Button(fenetre, text="Lancer Script 2", command=lambda: lancer_script(script2),
height=2, width=20)
btn3 = tk.Button(fenetre, text="Lancer Script 3", command=lambda: lancer_script(script3),
height=2, width=20)

btn1.pack(pady=10)
btn2.pack(pady=10)
btn3.pack(pady=10)

fenetre.mainloop()
```

Script en bash : Pour Linux, installer Zenity si non installé

```
#!/bin/bash

# Chemin vers les 3 scripts
SCRIPT1="$HOME/scripts/sauvegarde.sh"
SCRIPT2="$HOME/scripts/nettoyage.sh"
SCRIPT3="$HOME/scripts/.sh"
```

```
# Fonction pour Linux (zenity)
```

```
linux_launcher() {  
    choice=$(zenity --list --title="Lanceur de Scripts" \  
        --text="Cliquez pour exécuter un script :" \  
        --column="Action" --column="Description" \  
        "Script 1" "Mettre à jour et nettoyer" \  
        "Script 2" "Lancer la sauvegarde" \  
        "Script 3" "" \  
        --height=200 --width=400 --cancel-label="Quitter")  
  
    case "$choice" in  
        "Script 1") bash "$SCRIPT1" ;;  
        "Script 2") bash "$SCRIPT2" ;;  
        "Script 3") bash "$SCRIPT3" ;;  
        esac  
}
```

```
# Fonction pour macOS (osascript)
```

```
macos_launcher() {  
    choice=$(osascript <<EOF  
        choose from list {"Script 1", "Script 2", "Script 3"} \  
        with title "Lanceur de Scripts" \  
        with prompt "Cliquez pour exécuter un script :" \  
        OK button name "Exécuter" cancel button name "Quitter"  
EOF  
    )  
  
    case "$choice" in  
        "Script 1") bash "$SCRIPT1" ;;  
        "Script 2") bash "$SCRIPT2" ;;  
        "Script 3") bash "$SCRIPT3" ;;  
        esac  
}
```

```
# Détection de l'OS
```

```
if [[ "$OSTYPE" == "linux-gnu"* ]]; then  
    linux_launcher  
elif [[ "$OSTYPE" == "darwin"* ]]; then  
    macos_launcher  
else  
    echo "OS non supporté. Ce script fonctionne sous Linux et macOS uniquement."  
fi
```

Problème constaté sur VM : le lanceur de scripts ne fonctionne pas sur le réseau d'univ-nantes, vraisemblablement un problème de permissions ?
Fonctionne avec EduVPN cependant

-----rsink
-freeNX